Complete Guide with Examples for Microservices Architecture

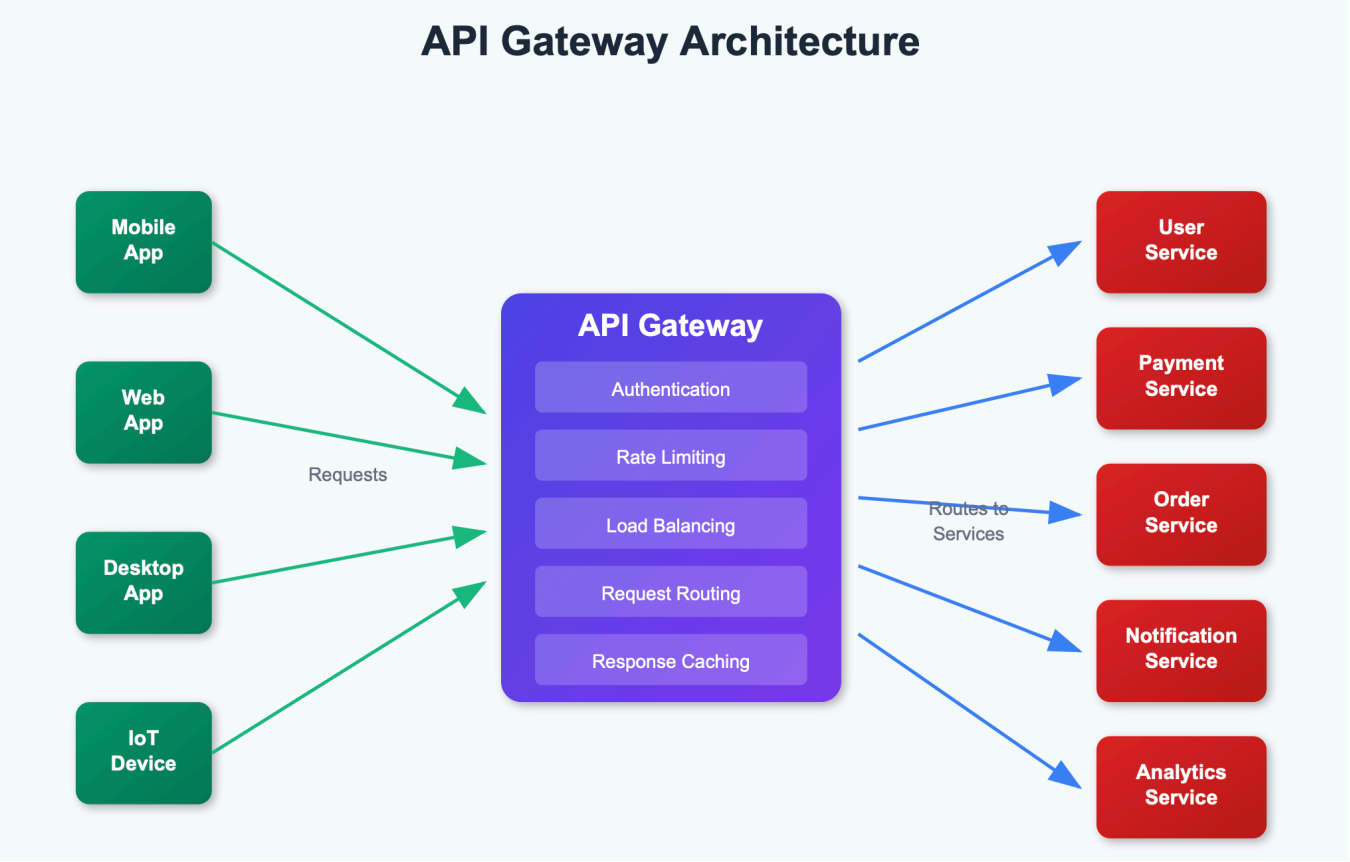
systemdesignschool.io/fundamentals/api-gateway

API Gateway

Now that we've covered the basics of API design, let's talk about API Gateway. In most microservices architecture, we have multiple services, and clients need to communicate with these services. Clients would need to know the addresses of all microservices and handle different protocols and data formats. This creates tight coupling between clients and services, making the system harder to maintain and evolve. API Gateway is a solution to this problem by providing a single entry point for all client requests. Additionally, API Gateway can help us implement common features like authentication, authorization, rate limiting, and monitoring.

What is an API Gateway?

An API Gateway is a server that acts as an entry point to a microservices architecture. It sits between clients and backend services, handling all incoming requests and routing them to the appropriate microservices. Think of it as a reverse proxy with additional capabilities specifically designed for API management.



API Gateway acts as a single entry point for all client requests

Why Do We Need an API Gateway?

In a microservices architecture, clients need to communicate with multiple services. Without an API Gateway, clients would need to:

- Know the addresses of all microservices
- Handle different protocols and data formats
- Implement authentication and authorization for each service
- Manage rate limiting and monitoring independently
- Deal with cross-cutting concerns like logging and security

This creates tight coupling between clients and services, making the system harder to maintain and evolve.

Key Features of an API Gateway

1. Request Routing

The API Gateway routes incoming requests to the appropriate backend services based on various criteria such as URL path, HTTP method, headers, or query parameters.

```
GET /api/users/123 → User Service
GET /api/orders/456 → Order Service
POST /api/payments → Payment Service
```

2. Load Balancing

API Gateways can distribute requests across multiple instances of the same service, providing built-in load balancing capabilities.

For a comprehensive understanding of load balancing algorithms, strategies, and implementation details, see our dedicated Load Balancer lesson.

3. Authentication and Authorization

Centralized authentication and authorization eliminate the need for each microservice to implement these features independently. This is where API Gateway becomes particularly powerful in modern system design.

- **Authentication**: Verify user identity (JWT tokens, API keys, OAuth)
- **Authorization**: Check if the authenticated user has permission to access specific resources

4. Rate Limiting and Throttling

Protect backend services from being overwhelmed by implementing rate limiting policies.

```
Per-user limits: 100 requests/minute
Per-IP limits: 1000 requests/hour
Service-specific limits: 10,000 requests/hour for search service
```

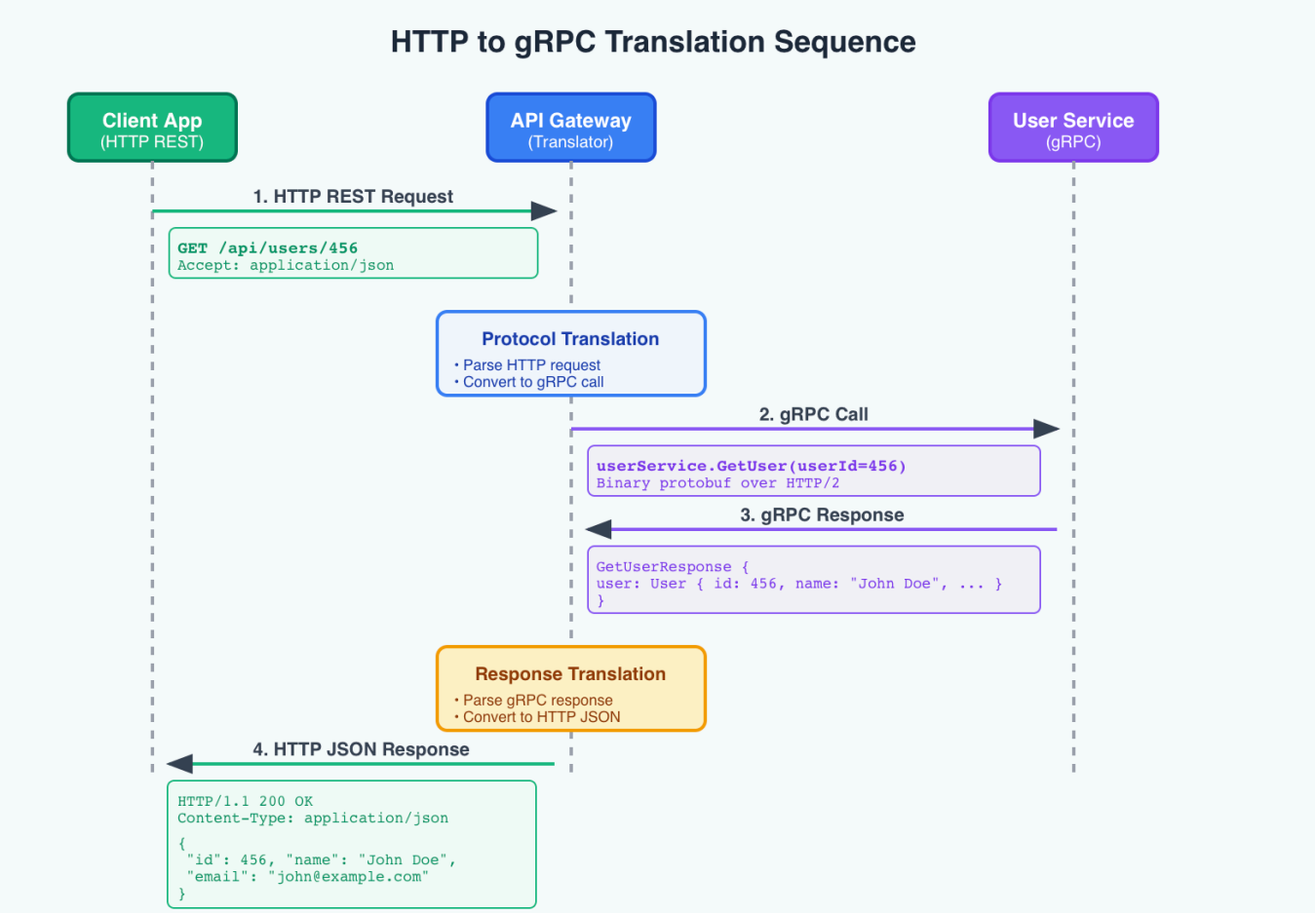
5. Protocol Translation

Convert between different protocols and data formats:

- HTTP to gRPC
- REST to GraphQL
- JSON to XML
- WebSocket connections

Example: HTTP to gRPC Translation

Many backend services use gRPC for high-performance internal communication, but clients expect simple HTTP REST APIs:




```
Client Request (HTTP REST):
GET /api/users/456

API Gateway translates to:
Backend Service (gRPC): userService.GetUser(userId=456)

API Gateway receives gRPC response and converts back to HTTP JSON:
{
  "id": 456,
  "name": "John Doe",
  "email": "john@example.com"
}
```

This allows:

- **Backend services** to use efficient gRPC for internal communication
- **Clients** to use familiar HTTP REST APIs
- **Teams** to choose optimal protocols without affecting client experience

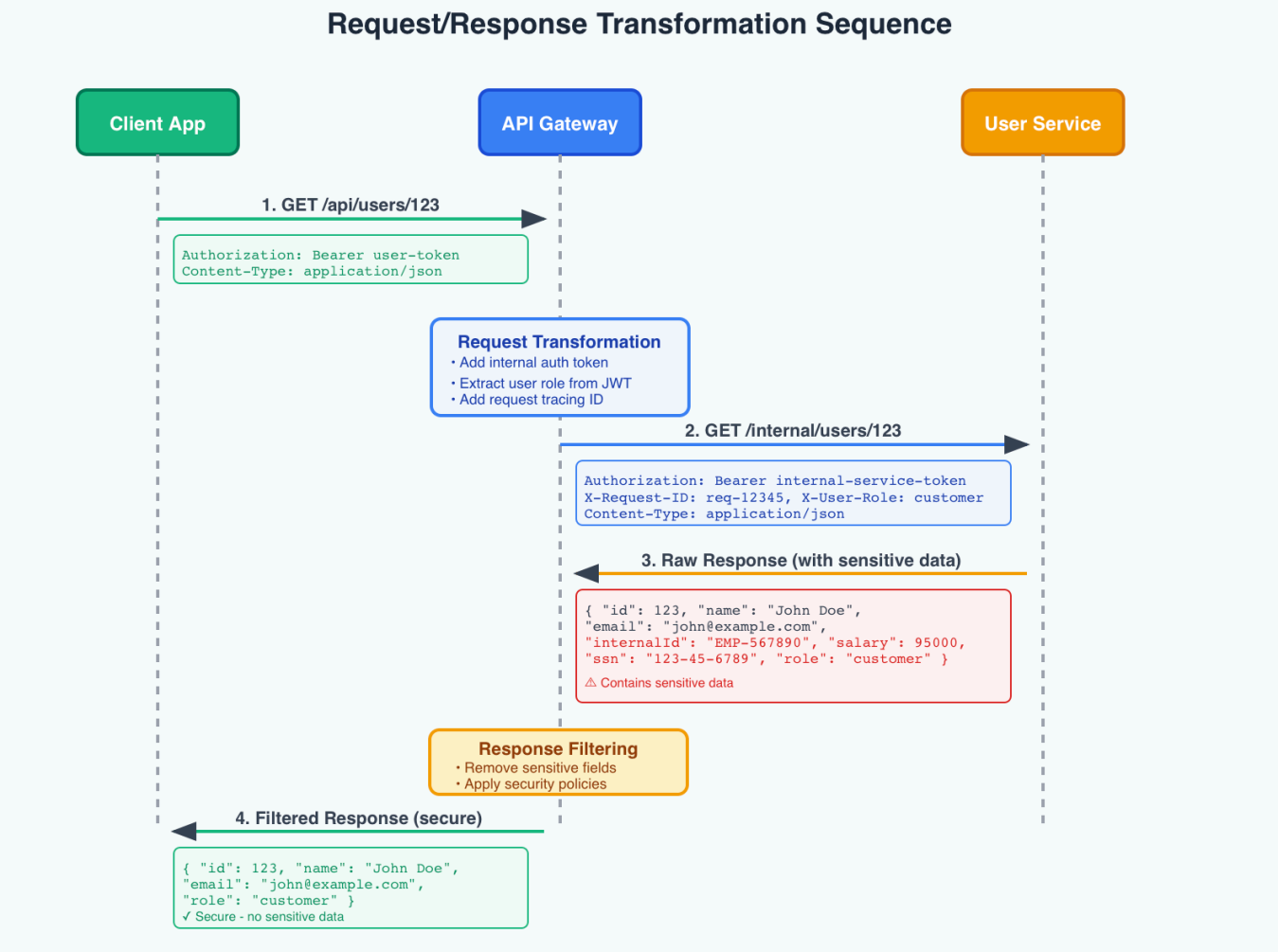
6. Request/Response Transformation

The API Gateway can modify requests and responses as they flow through it, acting as a translator between what clients send/expect and what backend services require/return.

Modify requests and responses as they pass through the gateway:

- Add authentication headers
- Transform response formats
- Aggregate data from multiple services
- Filter sensitive information

Example: Header Injection and Response Filtering




```
// Client Request GET /api/users/123 // API Gateway transforms request by adding headers const backendRequest = { url: '/internal/users/123', headers: { 'Authorization': 'Bearer internal-service-token', 'X-Request-ID': 'req-12345', 'X-User-Role': 'customer' // extracted from client JWT } }; // Backend Service Response { "id": 123, "name": "John Doe", "email": "john@example.com", "internalId": "EMP-567890", "salary": 95000, "ssn": "123-45-6789", "role": "customer" } // API Gateway transforms response (filters sensitive data) { "id": 123, "name": "John Doe", "email": "john@example.com", "role": "customer" }
```

This allows:

- **Adding authentication** without modifying backend services
- **Filtering sensitive data** (salary, SSN, internal IDs) from responses
- **Injecting request metadata** for tracing and monitoring
- **Standardizing response formats** across different services

7. Monitoring and Analytics

Centralized logging, metrics collection, and analytics for all API traffic:

- Request/response logs
- Performance metrics
- Error rates
- Usage analytics

Popular API Gateway Solutions

Cloud-Based

- **AWS API Gateway**: Fully managed service with auto-scaling
- **Google Cloud Endpoints**: Integrated with Google Cloud services
- **Azure API Management**: Enterprise-grade features and analytics

Open Source

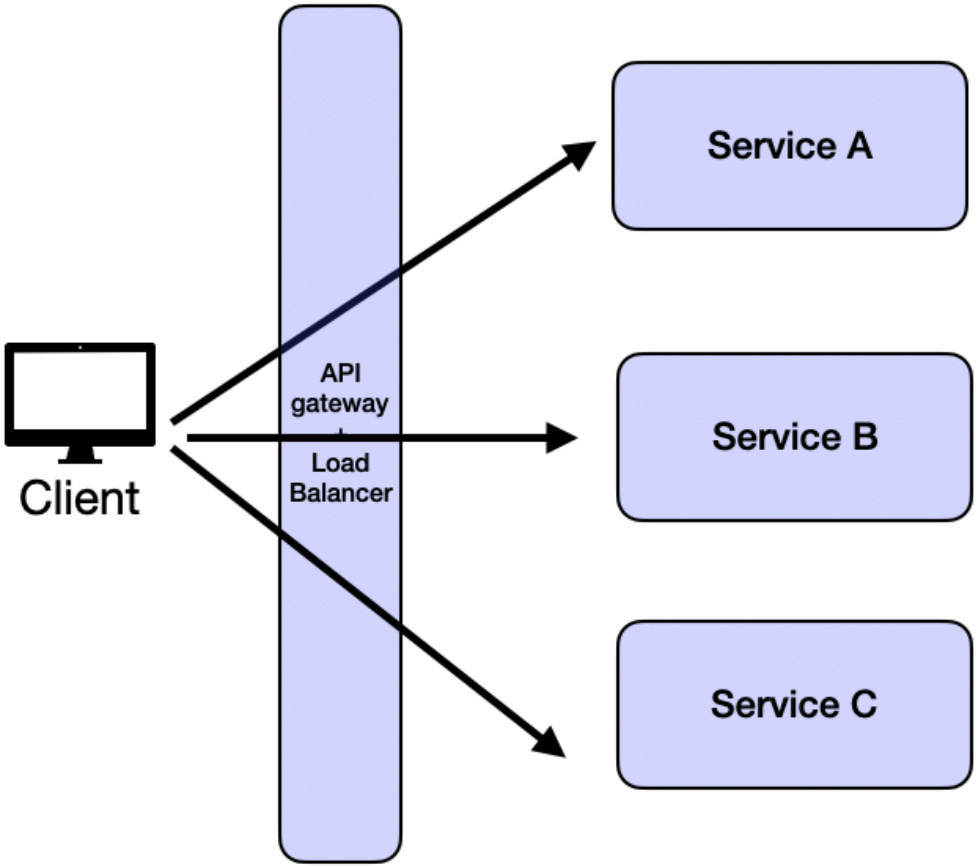
- **Kong**: Plugin-based architecture with extensive ecosystem
- **Zuul**: Netflix's API Gateway with Spring Cloud integration
- **Traefik**: Cloud-native with automatic service discovery
- **Envoy Proxy**: High-performance proxy with advanced load balancing

Self-Hosted

- **Nginx Plus**: Commercial version with API Gateway features
- **HAProxy**: High availability and performance focus
- **Ambassador**: Kubernetes-native API Gateway

API Gateway in System Design Interview

In design interviews, the focus is typically on designing for the business requirements and scalability. API gateway is commonly drawn as a single catch-all entity that is responsible for routing requests to the appropriate microservices while handing the authentication and authorization as well as rate limiting and monitoring.



If the interviewer asks you to deep dive on API gateway, you can use the information in this above sections to guide your design.